

校园信息聚合论坛系统（校园通）系统说明文档

姓名(组长)：朱子豪 学号：2312190118 班级：计科2301 github仓库地址：
https://github.com/likekiana/web_Final-project

一、项目背景及意义

1.1 项目背景

随着高等教育的发展，大学校园内的信息交流需求日益增长。然而，当前校园内的信息传播存在诸多问题：

- 信息分散：各类信息分布在不同平台，如微信群、QQ群、校园公众号等，学生难以全面获取
- 信息时效性差：很多重要信息（如讲座、活动、失物招领）未能及时传递给目标人群
- 信息难以沉淀：聊天群中的信息容易被淹没，无法长期保存和检索
- 缺乏结构化管理：信息缺乏分类和标签，查找困难

1.2 项目意义

校园信息聚合论坛系统（校园通）旨在解决上述问题，通过构建一个结构化、可搜索、可沉淀的校园社区平台，实现：

- 整合校园各类信息，打破信息孤岛
- 提高信息传播效率，确保重要信息及时触达目标用户
- 实现信息的长期保存和便捷检索
- 促进校园内不同群体之间的交流与互动
- 提升学生的校园生活质量和学习效率

二、项目用途

校园通系统主要用于以下场景：

1. **学习学术交流**：学生可以分享学习资料、考研/保研信息、学习经验，讨论学术问题
2. **校园生活服务**：提供生活攻略、失物招领、校内资讯、生活问答等功能
3. **二手交易**：方便学生进行教材书籍、数码产品、生活用品等的交易
4. **活动社交**：发布社团活动、比赛/竞赛信息，组织运动/娱乐组队
5. **实习就业**：提供实习信息发布、求职经验分享、企业宣讲会信息
6. **情感交流**：提供匿名树洞、表白墙等功能，满足学生的情感表达需求
7. **广告服务**：为校内商家、学生创业者提供广告发布平台
8. 具体功能还能根据情况实时调整添加

系统面向四类用户：普通学生用户、商户/广告用户、内容管理员和校园墙入驻者，满足不同角色的需求。

三、项目技术栈及架构

3.1 技术栈

类别	技术	版本	用途
----	----	----	----

类别	技术	版本	用途
后端框架	Django	5.x	Web服务器框架
数据库	MySQL	9.4	数据存储
后端API	Django REST Framework	3.15+	RESTful API开发
认证机制	JWT	-	用户认证与授权
前端框架	React	18.x	前端UI框架
前端构建工具	Vite	5.x	前端项目构建
UI组件库	Ant Design	5.x	前端UI组件
HTTP客户端	Axios	-	前后端数据交互
AI集成	DeepSeek API	-	智能问答、内容生成等AI功能

3.2 系统架构

系统采用前后端分离架构，主要包括以下层次：

- 1. **前端层**：使用React + Vite + Ant Design构建用户界面，负责与用户交互
- 2. **API层**：基于Django REST Framework构建RESTful API，处理前端请求
- 3. **业务逻辑层**：实现系统的核心业务逻辑，包括用户管理、内容管理、权限控制等
- 4. **数据层**：使用MySQL数据库存储系统数据，通过Django ORM进行数据库操作
- 5. **AI服务层**：集成DeepSeek API，提供智能问答、内容生成等AI功能

3.3 数据库设计

系统采用关系型数据库设计，核心表结构如下：

表名	描述	关系
users	用户信息	1:N (posts, comments, likes, reports, follows, favorites, browsing_history)
categories	板块分类	1:N (posts)
posts	帖子内容	N:1 (users, categories), 1:N (comments, likes, reports, favorites, browsing_history)
comments	评论信息	N:1 (users, posts), 1:N (likes, reports)
likes	点赞记录	N:1 (users, posts/comments)
reports	举报记录	N:1 (users, posts/comments)

表名	描述	关系
follows	用户关注	N:1 (users)
favorites	帖子收藏	N:1 (users, posts)
browsing_history	浏览历史	N:1 (users, posts)

核心表设计要点：

- 用户表（users）：包含用户基本信息、角色权限、隐私设置等字段
- 帖子表（posts）：支持多种帖子类型，包含媒体文件（图片、视频）存储
- 点赞表（likes）：支持对帖子和评论的点赞，确保用户对同一对象只能点赞一次
- 私信相关表：支持用户之间的私密交流，包括文字、图片、视频等多种消息类型

数据库优化：

- 为经常查询的字段创建索引，提高查询性能
- 使用外键约束保证数据完整性
- 实现数据的软删除，便于数据恢复
- 定期清理无用数据，优化数据库性能

四、项目关键技术及解决方案

1. 前后端分离架构

技术实现：前端使用React + Vite + Ant Design，后端使用Django + Django REST Framework，通过RESTful API进行数据交互。

解决方案：

- 前端通过axios调用后端API，实现数据的获取和更新
- 后端使用Django REST Framework构建标准化的RESTful API
- 使用JWT进行身份认证，确保API调用的安全性
- 统一API响应格式，包含success、message和data字段，便于前端统一处理

遇到的困难：

- 前后端数据格式不一致（前端使用camelCase，后端使用snake_case）

关键代码：

```
// 前端API服务文件配置 (frontend/src/services/api.js)
const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL || 'http://localhost:8000/api',
  timeout: 30000,
  headers: {
    'Content-Type': 'application/json'
  }
})
```

```
}  
})
```

2. JWT认证机制

技术实现：使用django-rest-framework-simplejwt库实现JWT认证，包括token获取、刷新和验证。

解决方案：

- 在后端配置JWT认证视图和权限类
- 前端在请求头中添加Authorization字段，包含JWT token
- 实现token过期自动刷新机制
- 在localStorage中存储token，管理登录状态

遇到的困难：

- token过期处理

关键代码：

```
// 认证上下文管理 (frontend/src/contexts/AuthContext.jsx)  
const login = async (credentials) => {  
  setLoading(true)  
  try {  
    const response = await authAPI.login(credentials)  
    if (response.success) {  
      // 保存token和用户角色到localStorage  
      localStorage.setItem('token', response.data?.token?.access)  
      localStorage.setItem('userRole', response.data?.user?.role)  
  
      // 更新状态  
      setUser(response.data?.user)  
      setIsAuthenticated(true)  
  
      return response  
    } else {  
      throw new Error(response.message || '登录失败')  
    }  
  } catch (error) {  
    console.error('Login failed:', error)  
    throw error  
  } finally {  
    setLoading(false)  
  }  
}
```

3. RESTful API设计

技术实现：遵循RESTful API设计原则，设计标准化的API端点，包括资源获取、创建、更新和删除操作。

解决方案：

- 使用Django REST Framework的ViewSet和Router自动生成API端点
- 实现分页、过滤、排序等功能，提高API的灵活性
- 统一API响应格式，确保前后端数据交互的一致性
- 添加API文档，便于前端开发和调试

遇到的困难：

- Django APPEND_SLASH机制导致的POST请求重定向问题
- 解决方案：统一API端点的URL格式，确保所有API调用都使用正确的尾随斜杠

4. 组件化开发

技术实现：使用React组件化开发，将系统拆分为多个可复用的组件，如PostList、PostDetail、CommentList等。

解决方案：

- 设计合理的组件结构，提高代码复用性和可维护性
- 使用Ant Design组件库，减少重复开发
- 实现组件间的通信，确保数据的正确传递
- 添加组件测试，提高代码质量

5. 状态管理

技术实现：使用React Context API进行全局状态管理，如登录状态、用户信息等。

解决方案：

- 创建AuthContext，管理用户的登录状态和基本信息
- 在组件中使用useContext钩子获取和更新状态
- 实现状态的持久化，如将用户角色保存到localStorage

6. AI功能集成

技术实现：集成DeepSeek API，实现智能问答、帖子创作辅助和智能搜索增强等功能。

解决方案：

- 创建ai_assistant应用，实现AI相关的后端功能
- 设计AI服务类，封装AI API调用逻辑
- 实现优雅降级方案，当AI服务不可用时自动切换到简单实现
- 前端创建AI助手组件，提供用户友好的AI功能界面

遇到的困难：

- sklearn库依赖问题

关键代码：

```
// AI功能API配置 (frontend/src/services/api.js)
export const aiAPI = {
  // 智能问答
  askAI: (data) => api.post('/ai/ask/', data),
  // 生成帖子标题
  generateTitle: (data) => api.post('/ai/generate-title/', data),
  // 生成帖子摘要
  generateSummary: (data) => api.post('/ai/generate-summary/', data),
  // 根据关键词扩展内容
  expandContent: (data) => api.post('/ai/expand-content/', data),
  // 增强搜索
  enhancedSearch: (params) => api.get('/ai/enhanced-search/', { params })
}
```

7. 多媒体支持

技术实现：支持图片和视频的上传和展示，包括帖子封面、内容图片和视频。

解决方案：

- 在Post模型中添加media_files字段，支持图片和视频URL数组
- 前端实现文件上传功能，支持多种媒体类型
- 根据文件类型动态渲染，图片使用img标签，视频使用video标签
- 在axios请求拦截器中添加FormData类型检测

遇到的困难：

- 视频上传和展示

关键代码：

```
// 文件上传处理 (frontend/src/services/api.js)
api.interceptors.request.use(
  config => {
    // 从localStorage获取token
    const token = localStorage.getItem('token')
    if (token) {
      config.headers.Authorization = `Bearer ${token}`
    }

    // 如果请求体是FormData, 移除默认的Content-Type, 让axios自动设置
    if (config.data instanceof FormData) {
      delete config.headers['Content-Type']
    }

    return config
  },
  error => {
    console.error('Request error:', error)
    return Promise.reject(error)
  }
)
```

```
}  
)
```

8. 私信功能实现

技术实现：实现用户之间的关注关系和私信交流功能。

解决方案：

- 后端实现关注/取消关注、获取关注列表、检查关注状态的API
- 实现私信发送、获取私信列表、标记已读的API
- 前端实现关注按钮和私信功能，支持直接与发帖人进行私密交流
- 实现私信抽屉组件，提供友好的聊天界面

遇到的困难：

- 私信发送时返回500错误
- 私信图片加载问题
- 私信消息左右区分显示

关键代码：

```
// 私信发送功能 (frontend/src/services/api.js)  
export const messageAPI = {  
  // 发送私信  
  sendMessage: (data) => {  
    // 如果包含文件，使用FormData  
    const isFormData = data.image || data.video;  
    if (isFormData) {  
      const formData = new FormData();  
      Object.keys(data).forEach(key => {  
        if (data[key]) {  
          formData.append(key, data[key]);  
        }  
      });  
      return api.post('/messages/', formData);  
    }  
    return api.post('/messages/', data);  
  },  
  // 其他私信相关API...  
}
```

9. 夜间模式实现

技术实现：支持深色主题和浅色主题的切换，提升用户在不同环境下的使用体验。

解决方案：

- 添加完整的深色主题CSS样式
- 使用Ant Design的ConfigProvider实现主题切换

- 实现主题切换开关，支持用户手动切换主题
- 将主题偏好保存到localStorage，实现主题的持久化

10. 性能优化

技术实现：优化系统性能，提高页面加载速度和响应性。

解决方案：

- 优化PostList组件的性能，减少不必要的渲染
- 实现组件的按需加载，提高初始页面加载速度
- 为频繁查询的字段添加索引，提高数据库查询性能
- 优化API调用，减少不必要的网络请求

11. 错误处理机制

技术实现：实现完善的错误处理机制，提高系统的可靠性和用户体验。

解决方案：

- 前端实现axios响应拦截器，统一处理API错误
- 为不同类型的错误显示友好的提示信息
- 后端实现统一的异常处理，返回标准化的错误响应
- 记录详细的错误日志，便于调试和监控

关键代码：

```
// 响应拦截器 (frontend/src/services/api.js)
api.interceptors.response.use(
  response => {
    return response.data
  },
  error => {
    console.error('Response error:', error)

    let errorMessage = '请求失败，请重试'
    if (error.response) {
      switch (error.response.status) {
        case 400:
          errorMessage = error.response.data.message || '请求参数错误'
          break
        case 401:
          errorMessage = '未授权，请重新登录'
          localStorage.removeItem('token')
          window.location.href = '/login'
          break
        case 403:
          errorMessage = '拒绝访问'
          break
        case 404:
          errorMessage = '请求的资源不存在'
          break
      }
    }
  }
)
```

```
        case 500:
            errorMessage = '服务器内部错误'
            break
        default:
            errorMessage = error.response.data.message || `请求失败, 状态码:
${error.response.status}`
    }
} else if (error.request) {
    errorMessage = '网络错误, 请检查网络连接'
}

console.error('API Error:', errorMessage)

return Promise.reject(error)
}
```

12. 主界面UI优化

技术实现：优化主界面布局和视觉效果，提升用户体验。

解决方案：

- 重新排序首页组件，将帖子和搜索功能放在AI问答上方
- 添加现代化的渐变横幅和卡片设计，增强视觉吸引力
- 优化组件间距和排版，提高页面的整体美观度
- 添加阴影、圆角和悬停效果，增强交互反馈
- 实现响应式设计，适配不同屏幕尺寸

13. 管理员后台实现

技术实现：实现完整的管理员后台，支持用户、内容、板块、广告和举报管理。

解决方案：

- 创建AdminDashboard组件，实现管理员后台的主框架
- 实现多个管理子页面：UserManagement、ContentManagement、CategoryManagement、AdManagement、ReportManagement
- 添加管理员后台导航菜单，支持权限控制
- 实现各种管理功能，包括查看、编辑、删除和审核操作

14. 敏感词过滤

技术实现：实现敏感词检测和替换功能，维护社区内容的健康性。

解决方案：

- 增强敏感词过滤机制，添加更多中文敏感词
- 优化敏感词替换算法，确保长词优先匹配
- 修复敏感词过滤中的逻辑错误，确保所有敏感词都能被正确检测和替换

15. 积分系统实现

技术实现：实现积分规则、积分记录和等级计算等功能，鼓励用户参与和贡献优质内容。

解决方案：

- 后端：创建points应用，实现积分规则、积分记录和等级计算的模型和服务
- 实现积分API端点，包括获取积分信息、积分记录、积分排行榜等功能
- 设计积分规则，包括注册、登录、发帖、评论、点赞等行为的积分奖励
- 前端：实现积分信息展示和积分排行榜

16. 反馈系统实现

技术实现：实现用户对系统的建议和Bug反馈功能，便于系统改进和问题修复。

解决方案：

- 后端：创建feedback应用，实现反馈模型、序列化器和视图
- 实现反馈API端点，包括创建反馈、获取反馈列表、处理反馈等功能
- 前端：实现反馈表单，支持提交反馈和查看反馈列表

17. 浏览历史功能

技术实现：记录用户浏览帖子的历史，支持查看和清空浏览记录。

解决方案：

- 后端实现浏览历史记录API，包括记录浏览、获取浏览列表和清空浏览记录
- 使用timezone.now()确保准确记录用户浏览时间
- 前端实现浏览历史列表展示，支持分页和排序
- 添加清除历史功能，支持用户一键清空浏览记录

18. 帖子点赞和评论统计功能

技术实现：实现帖子点赞和评论数量的自动更新和统计。

解决方案：

- 创建content/signals.py文件，添加信号处理函数
- 实现评论创建/删除时自动更新帖子评论数的信号处理
- 实现点赞创建/删除时自动更新帖子点赞数的信号处理

关键代码：

```
# 信号处理 (backend/content/signals.py)
@receiver(post_save, sender=Comment)
def update_post_comment_count_on_save(sender, instance, **kwargs):
    post = instance.post
    post.comment_count = Comment.objects.filter(post=post).count()
    post.save()

@receiver(post_delete, sender=Comment)
```

```
def update_post_comment_count_on_delete(sender, instance, **kwargs):
    post = instance.post
    post.comment_count = Comment.objects.filter(post=post).count()
    post.save()
```

19. 隐私设置完整实现

技术实现：实现全面的隐私设置功能，保护用户隐私。

解决方案：

- 后端：为User模型添加5个隐私设置字段：show_profile、allow_messages、allow_follow、show_posts、save_history
- 更新用户序列化器，添加隐私设置字段
- 前端：实现隐私设置表单，支持多种隐私选项的切换
- 实现隐私设置的保存和加载功能

20. 主题切换实现

技术实现：实现深色主题和浅色主题的切换功能，提升用户在不同环境下的使用体验。

解决方案：

- 使用Ant Design的ConfigProvider实现主题切换
- 添加完整的深色主题CSS样式，确保所有组件都能正确适配
- 实现主题切换开关，支持用户手动切换主题
- 将主题偏好保存到localStorage，实现主题的持久化
- 添加平滑的过渡效果，提升主题切换的视觉体验

五、负责的技术点及开发工作

1. 后端开发

- **用户系统：**实现用户注册、登录、个人资料管理等功能
- **内容管理系统：**开发帖子发布、编辑、删除、审核等功能
- **评论系统：**实现评论的发布、点赞、回复等功能
- **私信系统：**开发用户之间的私信交流功能，支持文字、图片、视频等多种消息类型
- **权限控制：**基于角色的权限控制，实现不同用户角色的权限管理

2. 数据库设计与优化

- 设计规范化的数据库模型，减少数据冗余
- 实现数据库索引优化，提高查询性能
- 开发数据库迁移脚本，管理数据库结构的变化
- 实现数据的软删除和恢复功能

3. AI功能集成

- 集成DeepSeek API，实现智能问答功能
- 开发AI帖子创作辅助功能，支持内容生成和扩展

- 实现AI智能搜索增强，提高搜索结果的相关性
- 设计AI服务类，封装AI API调用逻辑

4. 前端开发

- 开发用户界面组件，如帖子列表、帖子详情、评论列表等
- 实现用户交互功能，如点赞、收藏、关注等
- 开发管理后台，实现用户管理、内容管理、板块管理等功能
- 优化前端性能，提高页面加载速度和响应性

5. 系统测试与优化

- 编写单元测试和集成测试，确保系统功能的正确性
- 进行性能测试，优化系统性能
- 修复系统中的bug和问题
- 实现系统的错误处理和异常管理